DM

Java

Programmation :

- Distance Intervilles.
- Tri par sélection.
- Tri par propagation.

Mme Roubeau

COCO Mathis
GOUGEON Sterenn

Sommaire

Les objectifs à atteindre 3
Le schéma de la répartition du travail 4
Les outils utilisés 4
Les algorithmes 5
Les programmes 8
1- 8
2- La méthode de tri par sélection 10
3- La méthode de tri par propagation 11
Tests des programmes 12
Les difficultés rencontrées et solutions apportées 14

Les objectifs à atteindre

1. *Distance Intervilles.*

L'objectif est de pouvoir trouver la distance qui sépare une ville d'une autre dans les 10 villes proposées comme un GPS.

Il y a plusieurs règles :

- Que même si la ville de départ est plus basse dans la matrice on puisse retourner un résultat.
- L'utilisateur doit pouvoir recommencer autant de fois qu'il le veut.

Il faut donc un programme optimisé et qui permet de répondre à tous les critères.

2. *Méthodes de tri.*

Première partie : La méthode de tri par sélection.

L'objectif est de trier un vecteur par ordre croissant.

Il y a plusieurs règles :

- L'utilisateur doit pouvoir choisir le nombre de cases du vecteur et les remplir par les nombres qu'il veut.
- Il ne faut pas que le vecteur soit inférieur ou égal à 5.

Ce programme doit donc comprendre un contrôle de saisie pour avoir une taille réglementaire ainsi que bien trier les nombres.

Deuxième partie : La méthode de tri par propagation.

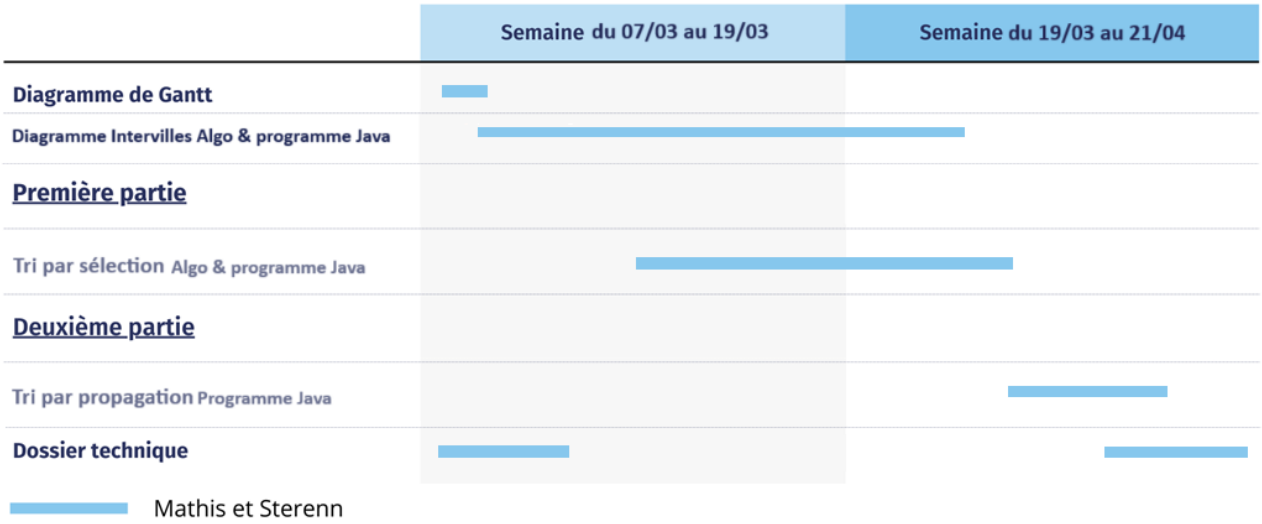
L'objectif est de trier le tableau en comparant chaque paire de nombres successifs pour savoir lequel est le plus grand et lequel est le plus petit, puis placer le plus grand à droite.

Il y a une règle :

- Il faut comprendre comment comparer les valeurs dans le tableau et ensuite permuter les valeurs pour trier le tableau.

Ce programme consiste en plusieurs échanges de valeurs et doit donc être bien optimisé pour ne pas faire plus de tours qu'il n'en faut déjà.

Le schéma de la répartition du travail



Les outils utilisés

Nous avons utilisé l'éditeur Eclipse pour les programmes Java ainsi que Word pour le compte rendu et Notepad++ pour les algorithmes.

Les algorithmes

Distance Intervilles :

```
Algorithme "Ville Distance"
Variables : numérique : idDépart = 1, idArrivée = 1, i;
                        CC = VDépart <-- "", VArrivée <-- "", rep
                        booléen : trouver = faux
Variables indicées : MDistance [][] = {{0, 996, 723, 890, 1286, 305, 564, 245, 1026, 884},
                                        {0, 0, 750, 104, 286, 711, 576, 747, 505, 543},
                                        {0, 0, 0, 668, 979, 593, 224, 515, 524, 905},
                                        {0, 0, 0, 0, 316, 607, 472, 645, 434, 467},
                                        {0, 0, 0, 0, 0, 890, 769, 938, 750, 400},
                                        {0, 0, 0, 0, 0, 0, 386, 106, 832, 559},
                                        {0, 0, 0, 0, 0, 0, 0, 348, 447, 681},
                                        {0, 0, 0, 0, 0, 0, 0, 0, 799, 665},
                                        {0, 0, 0, 0, 0, 0, 0, 0, 0, 901},
                                        {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}} de type numérique
Variables indicées : Villes [] = {"Brest", "Grenoble", "Lille", "Lyon", "Marseille", "Nantes",
                        "Paris", "Rennes", "Strasbourg", "Toulouse"} de type chaîne de caractères
Début
        Afficher "Bienvenue, les villes disponibles sont : "
        Pour i allant de 1 à 10
                Afficher "Ville ", (i+1), " : ", Villes[i]
        FinPour

        Répéter
                trouver <-- faux
                i = 1
                Tant que (trouver <-- faux)
                        Afficher "Veuillez saisir la ville de départ."
                        Saisir VDépart
                        Pour i allant de 1 à 10
                                Si (VDépart <-- Villes[i]) alors trouver <-- vrai
                                Fsi
                        FinPour
                        Si (VDépart <> trouver) alors Afficher "Veuillez saisir une ville de
                                                                départ correcte!"
                        Fsi
                FinTantque

                trouver <-- faux
                Tant que (trouver <-- faux)
                        Afficher "Veuillez saisir la ville d'arrivée."
                        Saisir VArrivée
                        Pour i allant de 1 à 10
                                Si (VArrivée <-- Villes[i]) alors trouver <-- vrai
                                Fsi
                        FinPour
                        Si (VArrivée <> trouver) alors Afficher "Veuillez saisir une ville
                                                                d'arrivée correcte!"
```

```
            Fsi
        FinTantque

        Pour i allant de 1 à 10
            Si (VDépart = Ville[i]) alors idDépart <-- i
            Fsi
            Si (VArrivée = Ville[i]) alors idArrivée <-- i
            Fsi
        FinPour

        Si (idDépart < idArrivée) alors Afficher "La distance entre ", VDépart, " et ", VArrivée, "
                                        est de ", MDistance[idDépart][idArrivée], " km."
                                    Afficher "La distance entre ", VDépart, " et ", VArrivée, "
                                        est de ", MDistance[idArrivée][idDépart], " km."
        Fsi

        Afficher "Voulez-vous recommencer?(Oui/Non)"
        Saisir rep
        Tant que (rep <> "Non") ET (rep <> "Oui")
            Afficher "Erreur de saisie! Veuillez entrer Oui ou Non."
            Saisir rep
        FinTantque
    Jusqu'à rep = "Non"
Fin
```

La méthode de tri par sélection :

```
Algorithme "Méthode de Tri"
Variables : numérique : i, k, j, N, min, ech;
Variables indicées : Tabtri[]

Début
	Afficher "Veuillez saisir le nombre de cases du tableau strictement supérieur à 5."
	Saisir N
		Tant que (N <--5)
			Afficher "Veuillez saisir le nombre de cases du tableau strictement supérieur à
5."
		FinTantque

		Pour i allant de 1 à N
			Tabtri[i] <-- "Veuillez saisir la valeur de la case " + (i + 1)+"."
		Finpour

	Afficher "Voici le vecteur à trier :"
		Pour i allant de 1 à N
			Afficher Tabtri[i] + " | "
		Finpour

		Pour i allant de 1 à N-1
			min <-- indicées
			Pour k allant de i+1 à N-1
				Si	(Tabtri[k] < Tabtri[min])
				Fsi
			Finpour
		Finpour
```

Les programmes

1- Distance Intervilles

```
package Villes;

public class Ville_Distance {

    public static void main(String[] args)
    {
        //Declaration des variables
        int idDepart = 0, idArrivee = 0;
        String VDepart = "", VArrivee = "", rep;
        boolean trouver = false;
        int i;
        int MDistance[][] = {{0, 996, 723, 890, 1286, 305, 564, 245, 1026, 884},
                             {0, 0, 750, 104, 286, 711, 576, 747, 505, 543},
                             {0, 0, 0, 668, 979, 593, 224, 515, 524, 905},
                             {0, 0, 0, 0, 316, 607, 472, 645, 434, 467},
                             {0, 0, 0, 0, 0, 890, 769, 938, 750, 400},
                             {0, 0, 0, 0, 0, 0, 386, 106, 832, 559},
                             {0, 0, 0, 0, 0, 0, 0, 348, 447, 681},
                             {0, 0, 0, 0, 0, 0, 0, 0, 799, 665},
                             {0, 0, 0, 0, 0, 0, 0, 0, 0, 901},
                             {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}};
        String Villes[] = {"Brest", "Grenoble", "Lille", "Lyon", "Marseille",
"Nantes", "Paris", "Rennes", "Strasbourg", "Toulouse"};

        System.out.println("Bienvenue, les villes disponibles sont : ");
        for (i=0; i<10;i++)
        {
            System.out.println("Ville " + (i+1) + " : " + Villes[i]);
        }
        System.out.println();

        do
        {
            //Controle de saisie de la ville de depart
            trouver = false;
            i=0;
            while (trouver == false)
```

```
            {
                VDepart = Saisie.lire_String("Veuillez saisir la ville de depart.");
                for (i=0; i<10;i++)
                {
                    if (VDepart.equalsIgnoreCase(Villes[i]))
                    {
                        trouver = true;
                    }
                }
                if (!trouver)
                {
                    System.out.println("Veuillez saisir une ville de depart
correcte!");
                    System.out.println();
                }
            }
            trouver = false;
            //Controle de saisie de la ville de d'arrivee
            while (trouver == false)
            {
                VArrivee = Saisie.lire_String("Veuillez saisir la ville d'arrivee");

                for (i=0; i<10;i++)
                {
                    if (VArrivee.equalsIgnoreCase(Villes[i]))
                    {
                        trouver = true;
                    }
                }
                if (!trouver)
                {
                    System.out.println("Veuillez saisir une ville d'arrivee
correcte!");
                    System.out.println();
                }
            }
            //On recupere le numero de la ville correspondante et on on instancie
idDepart
            for (i = 0; i < 10; i++) {
                if (VDepart.equalsIgnoreCase(Villes[i])) {
                    idDepart = i;
                }
```

```
            //On recupere le numero de la ville correspondante et on on instancie
idArrivee
                if (VArrivee.equalsIgnoreCase(Villes[i])) {
                    idArrivee = i;
                }
            }
            System.out.println();
            /*
            On donne la distance en mettant le numero de la ligne et de la colonne
correspondant
            a idDepart et a idArrivee si idDepart est inferieur a idArrivee et on
echance dans la matrice
            idDepart et idArrivee si idDepart est superieur a idArrivee.
            */
            if (idDepart < idArrivee)
             {
                System.out.println("La distance entre " + VDepart + " et " +
VArrivee + " est de " + MDistance[idDepart][idArrivee] + " km.");
             }
            else System.out.println("La distance entre " + VDepart + " et " +
VArrivee + " est de " + MDistance[idArrivee][idDepart] + " km.");


            //On demande à l'utilisateur s'il veut recommencer et on fait un
contrôle de saisie.
            System.out.println();
            rep = Saisie.lire_String("Voulez vous recommencez? (Oui/Non)");
            if ((!rep.equalsIgnoreCase("Non")) && (!rep.equalsIgnoreCase("Oui")))
            {
                rep = Saisie.lire_String("Erreur de saisie! Veuillez entrer Oui ou
Non");
            }
            System.out.println();
        }
        while (!rep.equalsIgnoreCase("Non"));
    }
}
```

2- La méthode de tri par sélection

```
package Tri;

public class Méthode_de_tri {

        public static void main(String[] args) {
                int i, k, j, N, min, ech;
    int Tabtri[];

    N = Saisie.lire_int("Veuillez saisir le nombre de cases du tableau strictement supérieur à 5.");
    while(N <= 5)
    {
       N = Saisie.lire_int("Veuillez saisir un nombre strictement supérieur à 5.");
    }
    Tabtri = new int[N];

    for (i = 0; i < N; i++)
    {
       Tabtri[i] = Saisie.lire_int("Veuillez saisir la valeur de la case " + (i + 1) + ".");
    }
    System.out.println();

    // Affichage du vecteur initial
    System.out.println("Voici le vecteur à trier :");
    for (i = 0; i < N; i++)
    {
       System.out.print(Tabtri[i] + " | ");
    }
    System.out.println();

    // Tri par sélection

    for (i = 0; i < N - 1; i++)
    {
        System.out.println();
        min = i;
       for (k = i + 1; k < N; k++)
       {
          if (Tabtri[k] < Tabtri[min])
          {
             min = k;
          }
       }
       // Échange des éléments
       ech = Tabtri[min];
       Tabtri[min] = Tabtri[i];
       Tabtri[i] = ech;

      //Affichage du vecteur après chaque étape
```

```
        System.out.println((i+1) +"e étape du tri :");
        for (j = 0; j < N; j++)
        {
            System.out.print(Tabtri[j] + " | ");
        }
        System.out.println();
    }

    System.out.println();
    // Affichage du vecteur trié
    System.out.println("Voici le vecteur trié :");
    for (i = 0; i < N; i++)
    {
        System.out.print(Tabtri[i] + " | ");
    }

  }
}
```

3- La méthode de tri par propagation

```
package Villes;
public class Tri_propagation{
   public static void main(String[] args) {
      int N, I, ech, Sauve;
      int[] tab;

      // Saisie de la taille du tableau
      N = Saisie.lire_int("Quelle est la taille de votre tableau?");

      tab = new int[N];

      // Remplissage du tableau
      for (I = 0; I < N; I++)
      {
         tab[I] = Saisie.lire_int("Veuillez saisir le contenu de la case de rang " + (I +1));
      }
      // Sauvegarde de la valeur de N pour la réutiliser après le tri
      Sauve = N;
      // Traitement
      while (N > 0) {
         I = 0;
         while (I < N)
         {
            if (tab[I] > tab[I + 1])
            {
               ech = tab[I];
               tab[I] = tab[I + 1];
               tab[I + 1] = ech;

               System.out.println("Tableau " + I);
               for (I = 0; I < Sauve; I++)
               {
                  System.out.print(tab[I] + " | ");
               }
               System.out.println();
            }
            I = I + 1;
         }
         N = N - 1;
      }
      System.out.println("Tableau trié ");
      //Affichage du tableau trié
      for (I = 0; I < Sauve; I++)
      {
         System.out.print(tab[I] + " | ");
      }
      System.out.println();
   }
}
```

Tests des programmes

Distance Intervilles :

```
Bienvenue, les villes disponibles sont :
Ville 1 : Brest
Ville 2 : Grenoble
Ville 3 : Lille
Ville 4 : Lyon
Ville 5 : Marseille
Ville 6 : Nantes
Ville 7 : Paris
Ville 8 : Rennes
Ville 9 : Strasbourg
Ville 10 : Toulouse

Veuillez saisir la ville de départ.
rennes
Veuillez saisir la ville d'arrivée
strasbourg

La distance entre rennes et strasbourg est de 799 km.

Voulez vous recommencez? (Oui/Non)
oui

Veuillez saisir la ville de départ.
Paris
Veuillez saisir la ville d'arrivée
grenoble

La distance entre Paris et grenoble est de 576 km.

Voulez vous recommencez? (Oui/Non)
non
```

Tri par sélection :

```
Veuillez saisir le nombre de cases du tableau strictement supérieur à 5.
6
Veuillez saisir la valeur de la case 1.
5
Veuillez saisir la valeur de la case 2.
4
Veuillez saisir la valeur de la case 3.
9
Veuillez saisir la valeur de la case 4.
2
Veuillez saisir la valeur de la case 5.
6
Veuillez saisir la valeur de la case 6.
1

Voici le vecteur à trier :
5 | 4 | 9 | 2 | 6 | 1 |

1e étape du tri :
1 | 4 | 9 | 2 | 6 | 5 |

2e étape du tri :
1 | 2 | 9 | 4 | 6 | 5 |

3e étape du tri :
1 | 2 | 4 | 9 | 6 | 5 |

4e étape du tri :
1 | 2 | 4 | 5 | 6 | 9 |

5e étape du tri :
1 | 2 | 4 | 5 | 6 | 9 |

Voici le vecteur trié :
1 | 2 | 4 | 5 | 6 | 9 |
```

Tri par propagation :

```
Quelle est la taille de votre tableau?
6
Veuillez saisir le contenu de la case de rang 1:
5
Veuillez saisir le contenu de la case de rang 2:
7
Veuillez saisir le contenu de la case de rang 3:
1
Veuillez saisir le contenu de la case de rang 4:
2
Veuillez saisir le contenu de la case de rang 5:
4
Veuillez saisir le contenu de la case de rang 6:
3
Le tableau avant le tri :
5 7 1 2 4 3
754321Le tableau trié :
1 2 3 4 5 7
```

Les difficultés rencontrées et solutions apportées

Nous n'avons pas eu de problèmes d'organisations car nous avons pu nous appelés régulièrement.

Les difficultés étaient plutôt techniques :

- Il a fallu bien comprendre chaque spécificité, pour notamment le tri par propagation, qui était une notion que nous avions du mal à assimiler au début. Cependant grâce à des recherches sur différents sites nous avons pu comprendre son fonctionnement.
- Pour les distances entre villes il a fallu comprendre comment remédier au problème quand l'utilisateur rentrait une ville de départ qui correspondait à une ligne inférieure de la ville d'arrivée. Il a juste fallu intervertir les lignes et colonnes dans la matrice.

Dans l'ensemble ces difficultés nous ont permis de grandement nous améliorer, surtout au niveau des compréhension des vecteurs et des matrices car il a fallu les manipuler et tester différentes choses pour avoir le résultat attendu.